

KAI Technical UI Pack V3.2

Persona non grata

Projekt: KAI — Kinetic Artificial Intelligence

Zweck: Technische UI-/Bot-/State-/Schema-Grundlage für Dashboard, Telegram, Agentenstatus, Signal Cards, Risiko-Warnungen und spätere Animationen

Version: V3.2

Status: Implementierungsnah / Claude-Code-ready

0. Zielbild

KAI soll technisch nicht als statisches Bild eingebaut werden, sondern als **zustandsgetriebene System-Persona**.

KAI reagiert auf:

- Marktanalyse
- Newsanalyse
- Social Buzz
- Signalqualität
- Risiko
- Papertrading
- Livetrading
- Systemfehler
- SENTR Security
- Watchdog Reports
- Architect-Empfehlungen
- Agentenstatus
- Datenqualität

KAI ist damit ein **UI-State-Host**, ein **Telegram-Renderer**, ein **Risk Narrator**, ein **Audit-Auslöser** und später ein **animierbarer Avatar**.

Leitregel:

KAI darf wild wirken. Die technische Umsetzung muss streng, typisiert, testbar und fail-closed sein.

1. Empfohlene Dateistruktur

```
src/  
  config/  
    kai_persona.yaml
```

```
kai_persona.schema.json

kai/
  types.ts
  constants.ts
  stateResolver.ts
  phraseEngine.ts
  riskGuards.ts
  auditMapper.ts
  assetMapper.ts
  i18n.ts

components/
  kai/
    KaiLiveWidget.tsx
    KaiAvatar.tsx
    KaiStatusBadge.tsx
    KaiCommentCard.tsx
    KaiSignalCard.tsx
    KaiWarningCard.tsx
    KaiAgentSummary.tsx
    KaiNextAction.tsx
    KaiAuditLink.tsx

telegram/
  kaiTelegramRenderer.ts
  kaiTelegramMenus.ts
  kaiTelegramTemplates.ts
  kaiTelegramGuards.ts

services/
  KaiPersonaService.ts
  KaiSignalService.ts
  KaiSystemService.ts
  KaiAuditService.ts

styles/
  kai.tokens.css

tests/
  kai/
    stateResolver.test.ts
    phraseEngine.test.ts
    riskGuards.test.ts
    assetMapper.test.ts
    i18n.test.ts
    telegramRenderer.test.ts
    kaiLiveWidget.test.tsx
    snapshot.test.tsx
```

2. TypeScript Core Types

```
export type KaiLanguage = "de" | "en";

export type KaiState =
  | "IDLE"
  | "ANALYSIS"
  | "SIGNAL"
  | "WARNING"
  | "SECURITY"
  | "ERROR"
  | "OFFLINE";

export type KaiSeverity =
  | "none"
  | "info"
  | "positive_watch"
  | "system"
  | "high"
  | "critical"
  | "unknown";

export type KaiTradingMode =
  | "WATCHLIST"
  | "PAPERTRADE"
  | "LIVETRADE"
  | "SIMULATION";

export type KaiDirection =
  | "LONG"
  | "SHORT"
  | "NEUTRAL"
  | "NO_TRADE";

export type KaiRiskLevel =
  | "LOW"
  | "MEDIUM"
  | "HIGH"
  | "CRITICAL";

export type KaiDataQuality =
  | "HIGH"
  | "MEDIUM"
  | "LOW"
  | "UNKNOWN";

export interface KaiStateDefinition {
  state: KaiState;
  priority: number;
  color: string;
```

```

    icon: string;
    animation: string;
    uiBehavior: string;
    severity: KaiSeverity;
    phrases: Record<KaiLanguage, string[]>;
}

export interface KaiRuntimeState {
    state: KaiState;
    severity: KaiSeverity;
    priority: number;
    statusLabel: string;
    color: string;
    icon: string;
    animation: string;
    comment: string;
    timestamp: string;
    source?: string;
    nextAction?: string;
}

export interface KaiSignalCardData {
    asset: string;
    mode: KaiTradingMode;
    direction: KaiDirection;
    confidence: number;
    risk: KaiRiskLevel;
    entry: string;
    stopLoss: string;
    dataBasis: string[];
    dataQuality: KaiDataQuality;
    timestamp: string;
    comment: string;
}

export interface KaiWarningCardData {
    target: string;
    problem: string;
    risk: KaiRiskLevel;
    action: string;
    timestamp: string;
    comment: string;
}

export interface KaiSecurityCardData {
    area: string;
    status: string;
    priority: "LOW" | "MEDIUM" | "HIGH" | "CRITICAL";
    lastCheck: string;
    result: string;
    nextStep: string;
}

```

```

    comment: string;
}

export interface KaiAgentStatus {
  agent: "SENTR" | "Watchdog" | "Architect" | "DALI" | "Neo" | "Satoshi";
  status: "OK" | "WARNING" | "ERROR" | "OFFLINE" | "UNKNOWN";
  summary: string;
  priority: number;
  timestamp: string;
}

export interface KaiLiveWidgetProps {
  runtimeState: KaiRuntimeState;
  lastSignal?: KaiSignalCardData;
  lastWarning?: KaiWarningCardData;
  agentStatuses?: KaiAgentStatus[];
  compact?: boolean;
  language?: KaiLanguage;
  onOpenAuditLog?: () => void;
  onOpenDetails?: () => void;
}

```

3. State Priority Resolver

3.1 Regel

KAI darf nie einen kritischen Zustand kosmetisch überschreiben.

Priorität:

```
ERROR > WARNING > SIGNAL > SECURITY > ANALYSIS > IDLE > OFFLINE
```

3.2 Beispielimplementierung

```

import type { KaiRuntimeState, KaiState } from "../types";

const STATE_PRIORITY: Record<KaiState, number> = {
  ERROR: 100,
  WARNING: 90,
  SIGNAL: 80,
  SECURITY: 70,
  ANALYSIS: 50,
  IDLE: 10,
  OFFLINE: 0,
};

```

```

export function resolveKaiState(states: KaiRuntimeState[]): KaiRuntimeState {
  if (!states.length) {
    return createFallbackState("OFFLINE", "Kein Signal. Keine Verbindung.");
  }

  return [...states].sort((a, b) => {
    const priorityA = STATE_PRIORITY[a.state] ?? -1;
    const priorityB = STATE_PRIORITY[b.state] ?? -1;
    return priorityB - priorityA;
  })[0];
}

export function createFallbackState(state: KaiState, comment: string):
KaiRuntimeState {
  const priority = STATE_PRIORITY[state] ?? 0;

  return {
    state,
    severity: state === "ERROR" ? "critical" : "unknown",
    priority,
    statusLabel: state,
    color: state === "OFFLINE" ? "#64748B" : "#FF1744",
    icon: state === "OFFLINE" ? "kai_offline" : "kai_error",
    animation: state === "OFFLINE" ? "static_fade" : "error_screen_tear",
    comment,
    timestamp: new Date().toISOString(),
    source: "fallback",
  };
}

```

4. Fail-Closed-Regeln

4.1 Grundsatz

Wenn KAI etwas nicht sicher bewerten kann, darf die UI niemals so tun, als sei alles in Ordnung.

4.2 Regeln

```

export function failClosedState(reason: string): KaiRuntimeState {
  return {
    state: "ERROR",
    severity: "critical",
    priority: 100,
    statusLabel: "ERROR",
    color: "#FF1744",
    icon: "kai_error",
  };
}

```

```

    animation: "error_screen_tear",
    comment: `Da knirscht etwas im Maschinenraum. ${reason}`,
    timestamp: new Date().toISOString(),
    source: "fail_closed_guard",
    nextAction: "System prüfen und Audit-Log öffnen.",
  };
}

```

Fail-closed auslösen bei:

- ungültigem State
- fehlender Datenbasis für Livetrading
- Confidence außerhalb 0–100
- fehlender Stop-Loss-Logik bei Livetrading
- kritischem Risiko
- Low Data Quality
- kaputter Telegram-Formatierung
- fehlender Systemquelle bei kritischen Alerts

5. Risk Guards für Trading

5.1 Zweck

KAI darf keine ungesicherten Livetrade-Aktionen durchlassen.

5.2 Beispielimplementierung

```

import type { KaiSignalCardData } from "../types";

export interface KaiGuardResult {
  allowed: boolean;
  reasons: string[];
}

export function validateSignalForLivetrade(signal: KaiSignalCardData):
KaiGuardResult {
  const reasons: string[] = [];

  if (signal.mode !== "LIVETRADE") {
    return { allowed: true, reasons: [] };
  }

  if (signal.risk === "CRITICAL") {
    reasons.push("Critical Risk blockiert Livetrading.");
  }

  if (signal.dataQuality === "LOW" || signal.dataQuality === "UNKNOWN") {

```

```

    reasons.push("Datenqualität reicht für Livetrading nicht aus.");
}

if (!signal.stopLoss || signal.stopLoss.toLowerCase().includes("wartet")) {
    reasons.push("Stop-Loss-Logik fehlt oder ist nicht bestätigt.");
}

if (!signal.dataBasis.length) {
    reasons.push("Datenbasis fehlt.");
}

if (signal.confidence < 0 || signal.confidence > 100) {
    reasons.push("Confidence liegt außerhalb des erlaubten Bereichs 0-100.");
}

return {
    allowed: reasons.length === 0,
    reasons,
};
}

```

6. Phrase Engine

6.1 Ziel

KAI soll wiedererkennbar sprechen, aber nicht monoton immer denselben Satz ausgeben.

6.2 Regeln

- Sprache: Deutsch und Englisch
- Kurze Sätze
- Keine Beleidigungen
- Keine Gewinnversprechen
- Keine Fake-Sicherheit
- State-spezifische Tonalität
- Bei Risiko immer direkter

6.3 Beispielimplementierung

```

import type { KaiLanguage, KaiState } from "../types";

const PHRASES: Record<KaiState, Record<KaiLanguage, string[]>> = {
  IDLE: {
    de: ["Ich bin ruhig. Nicht offline.", "Standby heißt nicht schlafen."],
    en: ["I am quiet. Not offline.", "Standby does not mean sleeping."],
  },
  ANALYSIS: {

```



```

    de: ["Datenstrom stabil. Ich sehe ein Muster.", "Ich trenne Signal von Theater."],
    en: ["Data stream stable. I see a pattern.", "Separating signal from theater."],
  },
  SIGNAL: {
    de: ["Ich habe etwas gefunden.", "Das ist kein Rauschen. Das ist ein Signal."],
    en: ["I found something.", "That is not noise. That is a signal."],
  },
  WARNING: {
    de: ["Stopp. Das ist nicht sauber.", "Warnsignal. Das riecht nach Liquiditätsfalle."],
    en: ["Stop. This is not clean.", "Warning signal. Smells like a liquidity trap."],
  },
  SECURITY: {
    de: ["System sauber. Keine roten Kabel sichtbar.", "Ich prüfe, ob es bricht."],
    en: ["System clean. No red wires visible.", "I check if it breaks."],
  },
  ERROR: {
    de: ["Da knirscht etwas im Maschinenraum.", "Input kaputt. Output gestoppt."],
    en: ["Something is grinding in the machine room.", "Input broken. Output stopped."],
  },
  OFFLINE: {
    de: ["Kein Signal. Keine Verbindung.", "Ich bin nicht weg. Nur nicht verbunden."],
    en: ["No signal. No connection.", "I am not gone. Just disconnected."],
  },
};

export function getKaiPhrase(state: KaiState, language: KaiLanguage = "de", seed?: number): string {
  const phrases = PHRASES[state]?.[language] ?? PHRASES.ERROR[language];
  const index = typeof seed === "number" ? Math.abs(seed) % phrases.length : Math.floor(Math.random() * phrases.length);
  return phrases[index];
}

```

7. Asset Mapper

```

import type { KaiState } from "../types";

export interface KaiAssetSet {
  staticImage: string;
}

```

```

    animationGif: string;
    animationWebm: string;
    icon: string;
}

export const KAI_ASSETS: Record<KaiState, KaiAssetSet> = {
  IDLE: {
    staticImage: "/assets/kai/kai_idle.png",
    animationGif: "/assets/kai/kai_idle_loop.gif",
    animationWebm: "/assets/kai/kai_idle_loop.webm",
    icon: "kai_idle",
  },
  ANALYSIS: {
    staticImage: "/assets/kai/kai_analysis.png",
    animationGif: "/assets/kai/kai_analysis_loop.gif",
    animationWebm: "/assets/kai/kai_analysis_loop.webm",
    icon: "kai_analysis",
  },
  SIGNAL: {
    staticImage: "/assets/kai/kai_signal.png",
    animationGif: "/assets/kai/kai_signal_found.gif",
    animationWebm: "/assets/kai/kai_signal_found.webm",
    icon: "kai_signal",
  },
  WARNING: {
    staticImage: "/assets/kai/kai_warning.png",
    animationGif: "/assets/kai/kai_risk_detected.gif",
    animationWebm: "/assets/kai/kai_risk_detected.webm",
    icon: "kai_warning",
  },
  SECURITY: {
    staticImage: "/assets/kai/kai_security.png",
    animationGif: "/assets/kai/kai_security_scan.gif",
    animationWebm: "/assets/kai/kai_security_scan.webm",
    icon: "kai_security",
  },
  ERROR: {
    staticImage: "/assets/kai/kai_error.png",
    animationGif: "/assets/kai/kai_error_detected.gif",
    animationWebm: "/assets/kai/kai_error_detected.webm",
    icon: "kai_error",
  },
  OFFLINE: {
    staticImage: "/assets/kai/kai_offline.png",
    animationGif: "/assets/kai/kai_no_signal.gif",
    animationWebm: "/assets/kai/kai_no_signal.webm",
    icon: "kai_offline",
  },
};

```

8. CSS / Design Tokens

8.1 `kai.tokens.css`

```
:root {
  --kai-black: #05070A;
  --kai-anthracite: #111827;
  --kai-border: #1F2937;
  --kai-text: #EAF6FF;
  --kai-muted: #64748B;

  --kai-cyan: #00E5FF;
  --kai-blue: #2F7DFF;
  --kai-magenta: #FF2BD6;
  --kai-violet: #8B5CF6;
  --kai-green: #00FFA3;
  --kai-orange: #FF6B00;
  --kai-red: #FF1744;

  --kai-state-idle: #00B8D9;
  --kai-state-analysis: #00E5FF;
  --kai-state-signal: #FF2BD6;
  --kai-state-warning: #FF6B00;
  --kai-state-security: #00FFA3;
  --kai-state-error: #FF1744;
  --kai-state-offline: #64748B;

  --kai-radius-card: 18px;
  --kai-radius-pill: 999px;
  --kai-shadow-soft: 0 18px 50px rgba(0, 229, 255, 0.08);
  --kai-shadow-warning: 0 18px 50px rgba(255, 107, 0, 0.12);
  --kai-shadow-error: 0 18px 50px rgba(255, 23, 68, 0.14);
}

.kai-card {
  background: linear-gradient(135deg, rgba(5, 7, 10, 0.96), rgba(17, 24, 39, 0.92));
  border: 1px solid var(--kai-border);
  border-radius: var(--kai-radius-card);
  color: var(--kai-text);
  box-shadow: var(--kai-shadow-soft);
}

.kai-status-badge {
  border-radius: var(--kai-radius-pill);
  padding: 4px 10px;
  font-size: 12px;
  letter-spacing: 0.08em;
  text-transform: uppercase;
}
```

```

    font-weight: 700;
}

.kai-state-ANALYSIS { color: var(--kai-state-analysis); }
.kai-state-SIGNAL { color: var(--kai-state-signal); }
.kai-state-WARNING { color: var(--kai-state-warning); }
.kai-state-SECURITY { color: var(--kai-state-security); }
.kai-state-ERROR { color: var(--kai-state-error); }
.kai-state-OFFLINE { color: var(--kai-state-offline); }

```

9. React-Komponente: KaiLiveWidget

9.1 Minimaler Produktionsentwurf

```

import React from "react";
import type { KaiLiveWidgetProps } from "../../kai/types";
import { KAI_ASSETS } from "../../kai/assetMapper";

export function KaiLiveWidget({
  runtimeState,
  lastSignal,
  lastWarning,
  agentStatuses = [],
  compact = false,
  language = "de",
  onOpenAuditLog,
  onOpenDetails,
}: KaiLiveWidgetProps) {
  const assets = KAI_ASSETS[runtimeState.state] ?? KAI_ASSETS.ERROR;

  return (
    <section className={`kai-card ${compact ? "kai-card--compact" : "kai-card--full"}`} >
      <header className="kai-card__header">
        <div className="kai-card__identity">
          <div className="kai-card__title">KAI LIVE</div>
          <div className="kai-card__subtitle">Persona non grata</div>
        </div>

        <span
          className={`kai-status-badge kai-state-${runtimeState.state}`}
          aria-label={`KAI status ${runtimeState.statusLabel}`}
        >
          {runtimeState.statusLabel}
        </span>
      </header>
    </section>
  );
}

```

```

<div className="kai-card__body">
  <div className="kai-avatar-shell">
    <video
      src={assets.animationWebm}
      poster={assets.staticImage}
      autoPlay
      muted
      loop
      playsInline
      className="kai-avatar"
    />
  </div>

  <div className="kai-card__content">
    <p className="kai-comment">{runtimeState.comment}</p>
    <p className="kai-timestamp">{new
Date(runtimeState.timestamp).toLocaleString(language === "de" ? "de-DE" :
"en-US")}</p>

    {!compact && lastSignal && (
      <div className="kai-mini-card kai-mini-card--signal">
        <strong>Last Signal</strong>
        <span>{lastSignal.asset} {lastSignal.direction}
{lastSignal.confidence}% Risk {lastSignal.risk}</span>
      </div>
    )}

    {!compact && lastWarning && (
      <div className="kai-mini-card kai-mini-card--warning">
        <strong>Last Warning</strong>
        <span>{lastWarning.target} {lastWarning.risk}
{lastWarning.problem}</span>
      </div>
    )}

    {!compact && agentStatuses.length > 0 && (
      <div className="kai-agent-row">
        {agentStatuses.slice(0, 6).map((agent) => (
          <span key={agent.agent} className={`kai-agent kai-agent--$
{agent.status}`}>
            {agent.agent}: {agent.status}
          </span>
        ))}
      </div>
    )}
  </div>
</div>

<footer className="kai-card__footer">
  {runtimeState.nextAction && <span>{runtimeState.nextAction}</span>}
  <div className="kai-card__actions">

```

```

        <button type="button" onClick={onOpenDetails}>Details</button>
        <button type="button" onClick={onOpenAuditLog}>Audit</button>
    </div>
</footer>
</section>
);
}

```

9.2 UI-Anforderung

Die Komponente muss:

- in Dark Mode stark wirken
- in Light Mode lesbar bleiben
- mobilefähig sein
- mit WebM bevorzugt arbeiten
- bei fehlendem WebM auf PNG fallen
- keine Fake-Daten anzeigen
- kritische Zustände klar priorisieren

10. Telegram Renderer

10.1 Signal Card Renderer

```

import type { KaiSignalCardData, KaiLanguage } from "../kai/types";

function escapeMarkdown(value: string): string {
    return value.replace(/([_*(\[ ]()~`>#+\-=|{ } . !])/g, "\\$1");
}

export function renderKaiSignalCard(signal: KaiSignalCardData, language:
KaiLanguage = "de"): string {
    const title = signal.mode === "WATCHLIST" ? "KAI SIGNAL // WATCHLIST" :
`KAI SIGNAL // ${signal.mode}`;

    if (language === "en") {
        return `*${title}*\n\n` +
            `Asset: ${escapeMarkdown(signal.asset)}\n` +
            `Direction: ${signal.direction}\n` +
            `Confidence: ${signal.confidence}%\n` +
            `Risk: ${signal.risk}\n` +
            `Entry: ${escapeMarkdown(signal.entry)}\n` +
            `Stop-Loss: ${escapeMarkdown(signal.stopLoss)}\n` +
            `Data Basis: ${escapeMarkdown(signal.dataBasis.join(", "))}\n` +
            `Data Quality: ${signal.dataQuality}\n` +
            `Time: ${escapeMarkdown(signal.timestamp)}\n\n` +
            `Comment:\n,${escapeMarkdown(signal.comment)}“`;
    }
}

```

```

}

return `*${title}*\\n\\n` +
  `Asset: ${escapeMarkdown(signal.asset)}\\n` +
  `Richtung: ${signal.direction}\\n` +
  `Confidence: ${signal.confidence}%\\n` +
  `Risiko: ${signal.risk}\\n` +
  `Entry: ${escapeMarkdown(signal.entry)}\\n` +
  `Stop-Loss: ${escapeMarkdown(signal.stopLoss)}\\n` +
  `Datenbasis: ${escapeMarkdown(signal.dataBasis.join(", "))}\\n` +
  `Datenqualität: ${signal.dataQuality}\\n` +
  `Zeit: ${escapeMarkdown(signal.timestamp)}\\n\\n` +
  `Kommentar:\\n,${escapeMarkdown(signal.comment)}“`;
}

```

10.2 Warning Card Renderer

```

import type { KaiWarningCardData, KaiLanguage } from "../kai/types";

export function renderKaiWarningCard(warning: KaiWarningCardData, language:
KaiLanguage = "de"): string {
  if (language === "en") {
    return `*KAI WARNING // RISK*\\n\\n` +
      `Target: ${escapeMarkdown(warning.target)}\\n` +
      `Problem: ${escapeMarkdown(warning.problem)}\\n` +
      `Risk: ${warning.risk}\\n` +
      `Action: ${escapeMarkdown(warning.action)}\\n` +
      `Time: ${escapeMarkdown(warning.timestamp)}\\n\\n` +
      `Comment:\\n,${escapeMarkdown(warning.comment)}“`;
  }

  return `*KAI WARNING // RISK*\\n\\n` +
    `Asset/System: ${escapeMarkdown(warning.target)}\\n` +
    `Problem: ${escapeMarkdown(warning.problem)}\\n` +
    `Risiko: ${warning.risk}\\n` +
    `Aktion: ${escapeMarkdown(warning.action)}\\n` +
    `Zeit: ${escapeMarkdown(warning.timestamp)}\\n\\n` +
    `Kommentar:\\n,${escapeMarkdown(warning.comment)}“`;
}

```

11. Telegram Menüs

```

export const KAI_TELEGRAM_MAIN_MENU_DE = {
  title: "KAI // CONTROL PANEL",
  buttons: [
    ["Markt scannen", "Signale anzeigen"],

```

```

    ["Risiko prüfen", "Portfolio prüfen"],
    ["Papertrading", "Livetrading"],
    ["Simulation", "News Radar"],
    ["Social Buzz", "Watchdog Report"],
    ["SENTR Security", "Einstellungen"],
  ],
};

export const KAI_TELEGRAM_MAIN_MENU_EN = {
  title: "KAI // CONTROL PANEL",
  buttons: [
    ["Scan Market", "Show Signals"],
    ["Check Risk", "Check Portfolio"],
    ["Paper Trading", "Live Trading"],
    ["Simulation", "News Radar"],
    ["Social Buzz", "Watchdog Report"],
    ["SENTR Security", "Settings"],
  ],
};

```

Telegram-Menüregeln:

- Menü jederzeit abrufbar
- klare Button-Zeilen
- keine ASCII-Tabelle
- keine kaputten Umlaute
- keine unnötigen Emojis
- kein unstrukturierter Textblock
- Livetrade immer mit Bestätigung

12. Audit-Log-Modell

12.1 Ziel

Jede wichtige KAI-Meldung muss nachvollziehbar bleiben.

12.2 Events

```

export type KaiAuditEventType =
  | "KAI_STATE_CHANGED"
  | "KAI_SIGNAL_RENDERED"
  | "KAI_WARNING_RENDERED"
  | "KAI_SECURITY_REPORT_RENDERED"
  | "KAI_LIVETRADE_BLOCKED"
  | "KAI_LIVETRADE_CONFIRMATION_REQUESTED"
  | "KAI_ERROR_STATE_TRIGGERED"
  | "KAI_AGENT_SUMMARY_RENDERED";

```



```
export interface KaiAuditEvent {
  id: string;
  type: KaiAuditEventType;
  timestamp: string;
  state: KaiState;
  severity: KaiSeverity;
  source: string;
  payload: Record<string, unknown>;
  message: string;
  correlationId?: string;
}
```

12.3 Audit-Regeln

Auditpflichtig sind:

- State-Wechsel
- Signal Cards
- Risk Warnings
- Security Reports
- Livetrade-Blockaden
- Livetrade-Bestätigungen
- Exchange-Ablehnungen
- Fehlerzustände
- Agentenberichte mit Priorität HIGH oder CRITICAL

13. JSON Schema für KAI Persona Config

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "KaiPersonaConfig",
  "type": "object",
  "required": ["kai"],
  "properties": {
    "kai": {
      "type": "object",
      "required": ["id", "name", "full_name", "motto", "version",
"state_machine"],
      "properties": {
        "id": { "type": "string", "const": "kai" },
        "name": { "type": "string" },
        "full_name": { "type": "string" },
        "motto": { "type": "string", "const": "Persona non grata" },
        "version": { "type": "string" },
        "language_default": { "type": "string", "enum": ["de", "en"] },
        "languages_supported": {
```

```

        "type": "array",
        "items": { "type": "string", "enum": ["de", "en"] },
        "minItems": 1
    },
    "state_machine": {
        "type": "object",
        "required": ["default_state", "priority_order", "states"],
        "properties": {
            "default_state": {
                "type": "string",
                "enum": ["IDLE", "ANALYSIS", "SIGNAL", "WARNING", "SECURITY",
"ERROR", "OFFLINE"]
            },
            "priority_order": {
                "type": "array",
                "items": {
                    "type": "string",
                    "enum": ["ERROR", "WARNING", "SIGNAL", "SECURITY",
"ANALYSIS", "IDLE", "OFFLINE"]
                }
            },
            "states": { "type": "object" }
        }
    }
}
}
}
}
}
}
}

```

14. Beispiel-Datenobjekte

14.1 Runtime State

```

{
    "state": "SIGNAL",
    "severity": "positive_watch",
    "priority": 80,
    "statusLabel": "SIGNAL FOUND",
    "color": "#FF2BD6",
    "icon": "kai_signal",
    "animation": "signal_found_pulse",
    "comment": "Ich habe etwas gefunden. Signal lebt. Risiko noch prüfen.",
    "timestamp": "2026-05-03T14:22:00.000Z",
    "source": "market_signal_engine",
    "nextAction": "Signal prüfen und Risiko validieren."
}

```

14.2 Signal Card

```
{
  "asset": "BTC/USDT",
  "mode": "WATCHLIST",
  "direction": "LONG",
  "confidence": 72,
  "risk": "MEDIUM",
  "entry": "Nicht bestätigt",
  "stopLoss": "Wartet auf Struktur",
  "dataBasis": ["News", "Volumen", "Chartstruktur"],
  "dataQuality": "MEDIUM",
  "timestamp": "2026-05-03T14:22:00.000Z",
  "comment": "Signal lebt. Einstieg noch nicht sauber. Ich warte auf Bestätigung statt blind reinzuspringen."
}
```

14.3 Warning Card

```
{
  "target": "ETH/USDT",
  "problem": "Volumen schwach, Social Hype zu hoch",
  "risk": "HIGH",
  "action": "Kein Entry ohne Bestätigung",
  "timestamp": "2026-05-03T14:25:00.000Z",
  "comment": "Zu viel Lärm. Zu wenig Fundament. Ich fasse das nicht ohne saubere Struktur an."
}
```

15. Teststrategie

15.1 Unit Tests

Pflichttests:

- `resolveKaiState` wählt höchste Priorität
- `resolveKaiState` fällt bei leerem Input auf OFFLINE
- ungültiger State triggert fail-closed
- Phrase Engine liefert Deutsch und Englisch
- Phrase Engine liefert keine leeren Sätze
- Asset Mapper liefert für jeden State ein Asset
- Livetrade Guard blockiert Critical Risk
- Livetrade Guard blockiert Low Data Quality
- Livetrade Guard blockiert fehlenden Stop-Loss
- Telegram Renderer escaped Markdown korrekt

- Telegram Renderer erhält Umlaute

15.2 Snapshot Tests

Snapshots für:

- KaiLiveWidget IDLE
- KaiLiveWidget ANALYSIS
- KaiLiveWidget SIGNAL
- KaiLiveWidget WARNING
- KaiLiveWidget SECURITY
- KaiLiveWidget ERROR
- KaiLiveWidget OFFLINE
- Telegram Signal Card DE
- Telegram Signal Card EN
- Telegram Warning Card DE
- Telegram Security Card DE

15.3 Safety Tests

Pflicht:

- keine Formulierung „sicherer Gewinn“
- keine Formulierung „garantiert“ bei Trading
- Livetrade nicht ohne explizite Bestätigung
- Missing data basis blockiert Livetrade
- Critical Risk blockiert Livetrade
- ERROR State überschreibt SIGNAL State

16. Claude-Code Masterprompt V3.2

Implement KAI Technical UI Pack V3.2 as a production-grade integration for the existing KAI system.

KAI identity:

- Name: KAI – Kinetic Artificial Intelligence
- Motto: Persona non grata
- Role: central visible AI media host for Dashboard, Telegram, signal commentary, risk warnings, security checks and agent summaries
- Personality: rogue cyberpunk AI media host, sharp, sarcastic, precise, intelligent, risk-first, security-aware

Core implementation tasks:

1. Add or update `config/kai\_persona.yaml` using the existing KAI persona configuration.
2. Add `kai\_persona.schema.json` and validate the config at startup.
3. Create typed TypeScript models for KAI states, runtime state, signal cards, warning cards, security cards, agent status and audit events.

4. Implement state resolver with strict priority:
ERROR > WARNING > SIGNAL > SECURITY > ANALYSIS > IDLE > OFFLINE.
5. Implement fail-closed behavior for invalid states, missing critical data and unsafe trading output.
6. Implement KAI phrase engine with DE/EN support.
7. Implement asset mapper for PNG/GIF/WebM state assets.
8. Implement `KaiLiveWidget` for the dashboard.
9. Implement compact and full display modes.
10. Implement KAI Telegram renderer for menus, signal cards, warning cards, security cards and exchange response cards.
11. Implement Livetrade safety guards:
 - block CRITICAL risk
 - block LOW or UNKNOWN data quality
 - block missing stop-loss logic
 - block missing data basis
 - require explicit user confirmation
12. Implement audit logging for all important KAI state transitions, signals, warnings, security reports, blocked trades and exchange responses.
13. Add CSS/Tailwind tokens for KAI colors and states.
14. Ensure German umlauts render correctly everywhere.
15. Avoid broken Markdown, ugly ASCII tables, excessive emojis, placeholder-only UI and fake live data.
16. Add unit tests, snapshot tests, i18n tests, Telegram rendering tests and safety tests.
17. Ensure invalid states fail closed into ERROR or OFFLINE, never into a false OK state.
18. Ensure Dashboard and Telegram wording stays consistent.

Acceptance criteria:

- KAI is implemented as an active state-driven UI persona, not static decoration.
- Dashboard shows KAI LIVE with real state, avatar/animation, comment, signal/warning summaries, agent statuses, next action and audit link.
- Telegram shows clean KAI // CONTROL PANEL menus and compact readable cards.
- Risk and ERROR states override all lower states.
- Livetrading cannot bypass safety gates.
- All important outputs are audit-logged.
- Tests pass.
- Implementation is typed, maintainable and security-first.

17. Akzeptanzcheckliste

Funktion

- ☐ `kai_persona.yaml` vorhanden
- ☐ Schema validiert Config
- ☐ State Resolver implementiert
- ☐ Fail-Closed implementiert

- ☐ Risk Guards implementiert
- ☐ Phrase Engine DE/EN implementiert
- ☐ Asset Mapper implementiert
- ☐ Audit Mapper implementiert
- ☐ Dashboard Widget implementiert
- ☐ Telegram Renderer implementiert

UI

- ☐ KAI LIVE sichtbar
- ☐ Compact Mode vorhanden
- ☐ Full Mode vorhanden
- ☐ Dark Mode sauber
- ☐ Light Mode lesbar
- ☐ Mobile responsive
- ☐ WebM mit PNG-Fallback
- ☐ Statusfarben korrekt
- ☐ Keine Fake-Livedaten

Telegram

- ☐ Menü jederzeit abrufbar
- ☐ Buttons sauber strukturiert
- ☐ Signal Cards lesbar
- ☐ Warning Cards lesbar
- ☐ Security Cards lesbar
- ☐ Exchange Responses strukturiert
- ☐ Umlaute korrekt
- ☐ Markdown sicher escaped

Sicherheit

- ☐ Critical Risk blockiert Livetrade
- ☐ Low Data Quality blockiert Livetrade
- ☐ Fehlender Stop-Loss blockiert Livetrade
- ☐ Fehlende Datenbasis blockiert Livetrade
- ☐ Livetrade benötigt Bestätigung
- ☐ Keine garantierten Gewinne
- ☐ ERROR überschreibt SIGNAL
- ☐ Audit-Logs vorhanden

18. Schlussdefinition

KAI Technical UI Pack V3.2 macht aus KAI keine Grafik, sondern ein **technisches Zustandswesen**.

Er sieht, kommentiert, warnt, blockiert, priorisiert und protokolliert.

KAI wird damit zur sichtbaren Oberfläche deines Systems:

- charismatisch im Dashboard
- präzise in Telegram
- streng bei Risiko
- hart bei Fehlern
- sauber im Audit
- kontrolliert in der Umsetzung

Persona non grata.

Nicht willkommen bei schlechten Daten.

Nicht bequem bei Risiko.

Nicht still, wenn das System knirscht.